KATEDRA INFORMATIKY
UNIVERZITA PALACKÉHO V OLOMOUCI
www.inf.upol.cz

Michal Krupka
krupka.inf.upol.cz
michal.krupka@upol.cz
17. listopadu 12, 771 46 Olomouc

Paradigmata programování 1 ⋄ poznámky k přednášce

10. Stromy

verze z 27. listopadu 2024

1 Struktura tečkových párů

Na minulých přednáškách jsme používali tečkové páry k budování datové struktury seznamu. Na této přednášce ukážeme jiné struktury, které lze pomocí párů vytvořit.

Začneme strukturou, ve které složka *Car* a *Cdr* páru hraje vždy roli schránky pro další strukturu (na rozdíl od seznamů, kde tuto roli hrála jen složka *Cdr*). Půjde o tzv. (**obecnou**) **strukturu tečkových párů**. V ní páry hrají roli schránek a ukládanými hodnotami mohou být jen atomy.

Samotný atom lze také chápat jako (atomickou) strukturu tečkových párů.

Ke strukturám tečkových párů není nutné říkat žádnou zvláštní teorii a je možné hned začít s řešením příkladů. Následující funkce např. hledá ve struktuře tečkových párů hodnotu splňující podmínku danou predikátem:

```
(defun struc-find-if (test struc)
  (if (atom struc)
      (if (funcall test struc)
          struc
        nil)
    (let ((res (struc-find-if test (car struc))))
      (if (null res)
          (struc-find-if test (cdr struc))
        res))))
```

Následující zjednodušení využívá zobecněné logiky Lispu a toho, že operátory `or` a `and` vracejí jako *Pravdu* hodnotu posledního vyhodnocovaného výrazu:

```
> (and 1 2)
2

> (or 1 2 nil)
2
```

```
(defun struc-find-if (test struc)
  (if (atom struc)
      (and (funcall test struc)
           struc)
    (or (struc-find-if test (car struc))
        (struc-find-if test (cdr struc)))))
```

Obě pomůcky (zobecněná logika a vlastnost operátorů `or` a `and`) jsou někdy ovšem kritizovány pro nevhodný styl.

2 Datová struktura strom

Strom jako datová struktura je dán konečnou množinou *uzlů* a zobrazením, které každému uzlu přiřadí množinu jeho *následníků*, což jsou opět uzly. Musí přitom platit, že

1. existuje právě jeden uzel, zvaný *kořen*, který není následníkem žádného uzlu,
2. každý jiný uzel je následníkem právě jednoho uzlu,
3. pro každý uzel u existuje $(k+1)$tice uzlů $(u_0, u_1, \ldots, u_k)$, kde u_0 je kořen, $u_k = u$ a pro libovolné dva po sobě jdoucí uzly je druhý následníkem prvního.

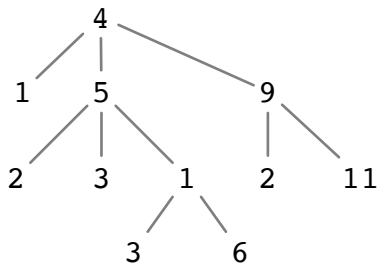
Je-li uzel v následníkem uzlu u, říkáme, že uzel u je *předchůdcem* uzlu v. Každý uzel kromě kořene (který nemá předchůdce) má právě jednoho předchůdce. Uzel, který nemá následníka, se nazývá *list*.

$k$tice $(u_0, u_1, \ldots, u_k)$ z bodu 3 se nazývá *cesta k uzlu u*, číslo k její *délka*. *Maximální cesta* je cesta k listu. *Výška stromu* je délka jeho nejdelší cesty.

Každý uzel stromu je kořenem jiného stromu, tzv. *podstromu*. Ten dostaneme tak, že se omezíme pouze na tento uzel, jeho následníky, následníky následníků atd.

Uzly stromu mají obvykle přiřazenu nějakou hodnotu. Množina následníků uzlu mívá často další strukturu: je omezena její velikost, uzly v ní jsou uspořádány, očíslovány a podobně.

Příklad. Stromy můžeme znázorňovat grafy jako například na tomto obrázku:



Místo uzlů stromu jsou na obrázku nakresleny jejich hodnoty.

V tomto stromu je jeden uzel s hodnotou 3 následníkem uzlu s hodnotou 5, druhý následníkem uzlu s hodnotou 1. Kořenem je uzel s hodnotou 4. Cesta k uzlu s hodnotou 6 obsahuje po řadě uzly s hodnotami 4, 5, 1, 6. Má délku 3. Je to maximální cesta, protože uzel s hodnotou 6 je list. Celý strom má výšku 3.

3 Reprezentace stromů pomocí seznamů

Uzel stromu můžeme v Lispu reprezentovat takto:

```
(hodnota . seznam-následníků)
```

Pokud lze z datové reprezentace uzlu zjistit i jeho následníky (jako v této reprezentaci), lze celý strom zadat jeho kořenem. Tak to budeme vždy dělat.

Příklad. Strom z předchozího příkladu můžeme tedy reprezentovat seznamem

```
(4 (1) (5 (2) (3) (1 (3) (6))) (9 (2) (11)))
```

Pro další použití si ho uložíme do pomocné proměnné.Konstruktor a selektory pro novou datovou strukturu:

```
(defun tree-node (val children)
  (cons val children))

(defun node-value (node)
  (car node))

(defun node-children (node)
  (cdr node))
```

4 Příklady

Následují základní příklady na práci se stromy.

Funkce `tree-find` má vrátit uzel zadaného stromu, jehož hodnota je rovna zadané hodnotě. V případě neúspěchu má vrátit `nil`. U každého stromu máme dvě možnosti: 1. hodnotu najdeme v kořenu, 2. hodnotu najdeme nebo nenajdeme v některém z následníků. Toto rozhodování provede samotná funkce `tree-find`.

Hledání v následnících znamená průchod seznamem následníků kořene. To dělá funkce `tree-list-find`.

```
(defun tree-find (value root)
  (if (eql (node-value root) value)
      root
    (tree-list-find value (node-children root))))

(defun tree-list-find (value roots)
  (if (null roots)
      nil
    (let ((res (tree-find value (car roots))))
      (if (null res)
          (tree-list-find value (cdr roots))
        res))))
```

Při operacích se stromy se často setkáme s tím, že operaci potřebujeme aplikovat na prvky seznamu stromů (jejich kořenů). Na to je často vhodné napsat pomocnou funkci podobně jako v uvedeném příkladě.

Hledání cesty k uzlu zadané hodnoty:

```
(defun tree-node-path (value root)
  (if (eql (node-value root) value)
      (list root)
    (let ((subpath (tree-list-node-path
                    value
                    (node-children root))))
      (if (null subpath)
          nil
        (cons root subpath)))))

(defun tree-list-node-path (value roots)
  (if (null roots)
      nil
    (let ((subpath (tree-node-path value (car roots))))
      (if (null subpath)
          (tree-list-node-path value (cdr roots))
        subpath))))
```

Následující funkce `tree-values-dfs` projde celý strom a vrátí seznam hodnot jeho uzlů. V zadání neurčujeme, v jakém pořadí mají hodnoty v seznamu být. To bude dáno tím, jak je funkce naprogramována: pro každý uzel rekurzivně zjistí seznam hodnot jeho podstromů a přidá k němu hodnotu uzlu. Zjištění seznamu hodnot podstromů se děje pomocí mapování a redukce.

```
(defun tree-values-dfs (root)
  (cons (node-value root)
        (foldr #'append
               (mapcar #'tree-values-dfs (node-children root))
               '())))
```

Funkci můžeme otestovat na našem stromu:

```
CL-USER 1 > (tree-values-dfs tree)
(4 1 5 2 3 1 3 6 9 2 11)
```

Na příkladu vidíme, proč se tomuto způsobu prohledávání stromu říká *prohledávání do hloubky* (*depth-first search*).

Nyní napíšeme funkci, která vrátí seznam všech hodnot ve stromu *prohledáním do šířky* (*breadth-first search*). Opět použijeme mapování a redukci.

```
(defun tree-values-bfs (root)
  (tree-list-values-bfs (list root)))

(defun tree-list-values-bfs (roots)
  (if (null roots)
      '()
    (append (mapcar #'node-value roots)
            (tree-list-values-bfs
             (foldr #'append
                    (mapcar #'node-children roots)
                    '()))))
```

Test, opět na našem stromu:

```
> (tree-values-bfs tree)
(4 1 5 9 2 3 1 2 11 3 6)
```

5 Binární stromy

Binární strom

Strom se nazývá *binární*, pokud má každý jeho uzel nejvýše dva následníky. Ty jsou označeny jako *levý* a *pravý*. Levý následník uzlu určuje jeho *levý podstrom*, pravý následník určuje *pravý podstrom*.

Uzel nemusí mít žádného následníka, nebo jen levého nebo pravého, nebo oba.

(Zjednodušený) binární vyhledávací strom

Hodnota každého uzlu je číslo. Pro každý uzel platí, že hodnoty všech uzlů v jeho levém podstromu jsou menší než hodnota uzlu, hodnoty všech uzlů v jeho pravém podstromu jsou větší.

Implementace

Změna: abychom jednoznačně určili, který následník je levý a který pravý, změníme reprezentaci uzlu na

```
(hodnota levý-následník pravý-následník)
```

Pokud následník neexistuje, bude na jeho pozici `nil`.

Implementace

Konstruktor:

```
(defun binary-tree-node (val left-child right-child)
  (list val left-child right-child))
```

Nové selektory:

```
(defun left-child (node)
  (cadr node))

(defun right-child (node)
  (caddr node))
```

Úprava funkce `node-children`:

```
(defun node-children (node)
  (remove nil (cdr node)))
```

Všechny předchozí funkce budou fungovat.

6 Reprezentace číselných množin binárními vyhledávacími stromy

Pokud je množina čísel reprezentována binárním vyhledávacím stromem s minimální možnou výškou (tzv. vyvážený binární vyhledávací strom), je hledání prvků v množině mnohem rychlejší než u reprezentace seznamem, kterou jsme probírali na minulé přednášce (místo lineární složitosti jde o složitost logaritmickou).

Následující dvě funkce implementují dvě základní akce pro množiny reprezentované binárním vyhledávacím stromem: testování, zda prvek leží v množině, a přidání prvku do množiny (přesněji řečeno vytvoření množiny, která obsahuje všechny prvky zadané množiny a jeden zadaný prvek navíc).

Nová verze funkce `elementp` zjišťuje, zda dané číslo leží v množině zadané binárním vyhledávacím stromem. Jako strom (parametr `tree`) přijímá i symbol `nil`.

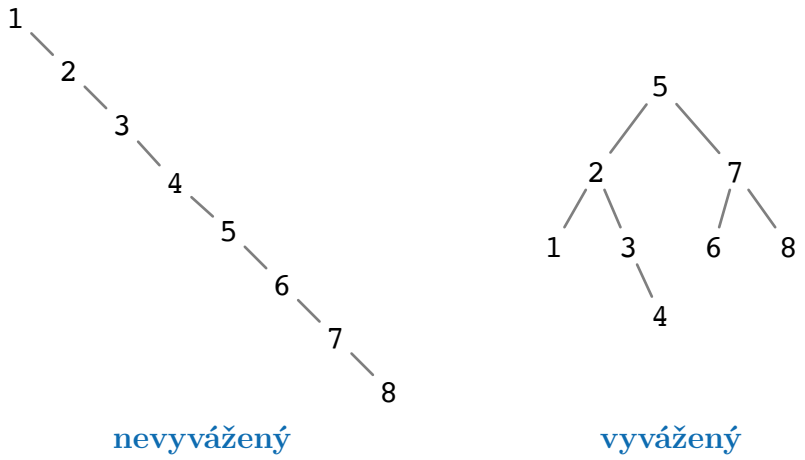
```
(defun bt-elementp (el tree)
  (and (not (null tree))
       (let ((val (node-value tree)))
         (or (= el val)
             (and (< el val)
                  (bt-elementp el (left-child tree)))
             (and (> el val)
                  (bt-elementp el (right-child tree)))))))
```

Výpočet bude tím rychlejší, čím bude mít strom menší výšku (počet rekurzivních volání je roven nejvýše délce cesty k nějakému listu).

Přidání prvku k množině reprezentované binárním vyhledávacím stromem realizuje funkce `my-adjoin`. Pokud prvek v množině už je, prvek podruhé nepřidává. Funkce se stará o to, aby nový prvek do stromu správně zařadila, tj. aby nový strom byl stále binární vyhledávací.

```
(defun bt-adjoin (elem tree)
  (if (null tree)
      (binary-tree-node elem nil nil)
    (let ((val (node-value tree))
          (left (left-child tree))
          (right (right-child tree)))
      (cond ((= elem val) tree)
            ((< elem val) (binary-tree-node val
                                            (bt-adjoin elem left)
                                            right))
            (t (binary-tree-node val
                                 left
                                 (bt-adjoin elem right)))))))
```

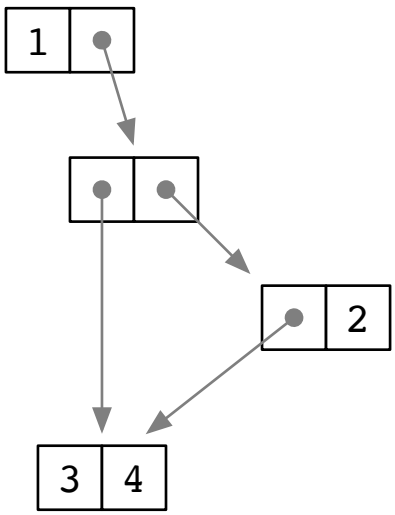
Jak už bylo řečeno, *vyvážený binární vyhledávací strom* (*balanced binary search tree*) je binární vyhledávací strom s minimální výškou. Platí, že výšky levého a pravého podstromu každého uzlu vyváženého binárního vyhledávacího stromu se liší nejvýše o 1.



Vyváženými binárními vyhledávacími stromy se nebudeme zabývat.

Otázky a úkoly na cvičení

1. Napište výraz, jehož hodnotou je struktura tečkových párů znázorněná krabičkovým znázorněním na tomto obrázku:



2. Napište funkci `flatten`, která vrátí seznam atomů ve struktuře tečkových párů v pořadí, ve kterém se v ní vyskytovaly v jejím zápisu:

```
> (bind l '(((1) 2 3 4) 5 (6 7)))
(((1) 2 3 4) 5 (6 7))
```

```
> (flatten l)
(1 2 3 4 5 6 7)
```

3. Napište funkci `deep-reverse`, která otočí daný seznam a všechny jeho podseznamy, pod-podseznamy atd.:

```
> (bind lst (list 1 (list 2 (list 3 4) 5) (list 6 7)))
(1 (2 (3 4) 5) (6 7))

> (deep-reverse lst)
((7 6) (5 (4 3) 2) 1)
```

4. Funkce `count` je napsána takto:

```
(defun count (x)
  (cond ((consp x) (+ 1 (count (car x)) (count (cdr x))))
        ((null x) 100)
        (t 0)))
```

   Jakou hodnotu má výraz `(count '(1 (2 3) 4 (5 . 6)))`?

5. Načrtněte obrázek stromu reprezentovaného seznamem `(a (c (b) (d)) (f (g (h)) (i)))`.

6. Napište výraz, který vytvoří strom reprezentovaný seznamem z minulého příkladu čistě pomocí konstruktoru `tree-node`.

7. Napište funkci `tree-sum`, která k danému stromu s číselnými hodnotami vrátí jejich součet.

8. Napište funkci `tree-maximal-paths`, která k danému stromu vrátí seznam všech cest k listům.

9. Napište funkci `tree-height`, která vrátí výšku daného stromu, aniž by zjišťovala cesty (tedy bez pomoci funkce `tree-maximal-paths`).

10. Napište výraz, který vytvoří reprezentaci vyváženého binárního vyhledávacího stromu z přednášky. Uzly vytvářejte výhradně pomocí konstruktoru `binary-tree-node`.

11. Vytvořte stejný binární strom z prázdného stromu opakovaným použitím funkce `my-adjoin`. Zkuste změnou pořadí přidávaných uzlů vytvořit ze stejných hodnot jiný strom.

12. Napište predikát `balancedp`, který zjistí, zda je daný binární vyhledávací strom vyvážený.

13. Napište funkci, která k danému číslu vrátí jeho uzel v binárním vyhledávacím stromu.